Machine Translated by Google

DesignCoder : Code d'interface utilisateur hiérarchique et auto-correcteur Génération avec de grands modèles de langage

Yunnong Chen
Faculté d'informatique et Technologie, Université du Zhejiang
Hangzhou, Chine

Shixian Ding
Faculté d'informatique et Technologie, Université du Zhejiang
Hangzhou, Chine

Yin Ying Zhang
Faculté d'informatique et Technologie, Université de Jilin
Jilin, Chine

Wenkai Chen
Faculté d'informatique et Technologie, Université du Zhejiang
Hangzhou, Chine

Jinzhou Du
Huawei Technologies Co., Ltd.
Shenzhen, Chine

Soleil Lingyun
Faculté d'informatique et Technologie, Université du Zhejiang
Hangzhou, Chine

Liuqing Chen
Faculté d'informatique et Technologie, Université du Zhejiang
Hangzhou, Chine

2506.13663v1 [cs.SE] 16 Jun 2025

Abstrait

Les modèles de langage multimodaux de grande taille (MLLM) ont simplifié le développement d'interfaces front-end en automatisant la génération de code. Cependant , ces modèles posent également des défis en matière de garantie de la qualité du code . Les approches existantes peinent à maintenir à la fois la cohérence visuelle et l'intégralité fonctionnelle des composants générés.

De plus, ils ne disposent pas de mécanismes permettant d'évaluer la fidélité et l'exactitude des pages rendues. Pour remédier à ces problèmes, nous proposons DesignCoder, un nouveau framework de génération de code automatisé, hiérarchique et auto-correcteur. Plus précisément, nous présentons Les chaînes de regroupement d'interface utilisateur, qui améliorent la capacité des modèles linéaires multiniveaux à comprendre et à prédire les hiérarchies d'interface utilisateur imbriquées complexes. Par la suite, DesignCoder utilise une approche hiérarchique de type diviser pour régner pour générer le code front-end. Enfin, nous intégrons une fonction d'autocorrection. mécanisme visant à améliorer la capacité du modèle à identifier et à corriger Erreurs dans le code généré. Évaluations approfondies sur un ensemble de données. des maquettes d'interface utilisateur collectées à la fois auprès des communautés open-source et Des projets industriels démontrent que DesignCoder surpasse les solutions de référence de pointe dans React Native, un framework d'interface utilisateur largement adopté. Notre méthode permet d'obtenir une amélioration des performances de 37,63 %, 9,52 % et 12,82 %. dans les métriques de similarité visuelle (MSE, CLIP, SSIM) et de manière significative améliore la similarité de la structure du code en termes de TreeBLEU, Con-tainer Match et Tree Edit Distance de 30,19 %, 29,31 % et 24,67 %. De plus, nous avons mené une étude auprès d'utilisateurs professionnels afin d'évaluer la qualité et la praticité du code généré. Les résultats indiquent que DesignCoder est conforme aux meilleures pratiques du secteur , démontrant une grande facilité d'utilisation, de lisibilité et de maintenance. Notre approche offre une solution efficace et pratique pour l'agilité Le développement front-end, permettant aux équipes de développement de se concentrer davantage sur… sur les fonctionnalités essentielles et l'innovation produit.

Format de référence ACM :
Yunnong Chen, Shixian Ding, Yinying Zhang, Wenkai Chen, Jinzhou Du, Lingyun Sun et Liuqing Chen. 2018. DesignCoder : un outil sensible à la hiérarchie et Génération de code d'interface utilisateur auto-correctif avec de grands modèles de langage. Dans les actes de [Veuillez vous assurer de saisir le titre correct de la conférence à partir de vos droits] Courriel de confirmation (Acronyme de la conférence : « XX »). ACM, New York, NY, États-Unis. 11 pages. https://doi.org/XXXXXXX.XXXXXXX

Auteur correspondant (chenlq@zju.edu.cn).

1 Introduction

La génération de code à partir d'interfaces graphiques a profondément transformé le secteur du développement front-end, accélérant le développement et réduisant les tâches répétitives pour les développeurs . Tirant parti des progrès de l'apprentissage profond et des vastes ensembles de données open source, de nombreux travaux se concentrent sur la traduction des représentations visuelles en code, depuis les croquis [22] et les wireframes [12] jusqu'aux interfaces graphiques. captures d'écran d'interface utilisateur haute fidélité [7]. Cependant, les applications industrielles à grande échelle Le développement impose des exigences strictes, notamment le respect de des cadres spécifiques et le besoin d'un code maintenable et accessible. Plusieurs plateformes de génération de code de niveau industriel, telles qu'Imgcook [4] et CodeFun [1] utilisent des maquettes comme entrée pour générer du code. Les maquettes créées par Sketch [5] et Figma [3] fournissent une représentation visuelle et incluent des métadonnées de conception essentielles, telles que les limites boîtes, hiérarchie et styles. Méthodes d'inspection de conception [8, 10, 11] Améliorer la qualité de la génération de code en détectant et en corrigeant les incohérences dans les maquettes de conception. Toutefois, des ajustements manuels supplémentaires restent nécessaires pour garantir pleinement l'exactitude du code généré. code.

Les récents progrès réalisés dans le domaine des modèles linéaires multiniveaux (MLLM) ont créé de nouvelles opportunités pour la génération de code à partir d'interfaces graphiques. De nombreuses études ont été proposées. pour améliorer les capacités de génération de code des MLLM [13, 15, 34] ou évaluer leurs performances [14, 25]. Xiao et al. ont proposé Proto-type2Code [30], qui combine la détection d'éléments et la mise en page informations permettant de générer du code HTML en exploitant les métadonnées de conception pour inciter les MLLM à générer du code CSS, ce qui s'avère prometteur résultats. DeclarUI [34] utilise des modèles de vision par ordinateur pour générer descriptions de fonctions et introduction de graphiques de transition de page, guidant MLLM pour générer une interface utilisateur d'application mobile avec une logique de saut.

Malgré ces progrès, plusieurs défis majeurs subsistent. Les MLLM existants peinent à maintenir une cohérence visuelle dans le code généré. Par exemple, le rendu présente souvent des éléments manquants. des éléments, des composants déformés et des agencements mal alignés, comme le montrent les cadres rouges de la figure 1. Cela est dû au fait que les MLLM manquent d'éléments, de composants déformés et de mises en page mal alignées, comme le soulignent les cadres rouges de la figure 1. compréhension approfondie des styles d'interface utilisateur détaillés, conduisant à une fidélité moindre

Machine Translated by Google

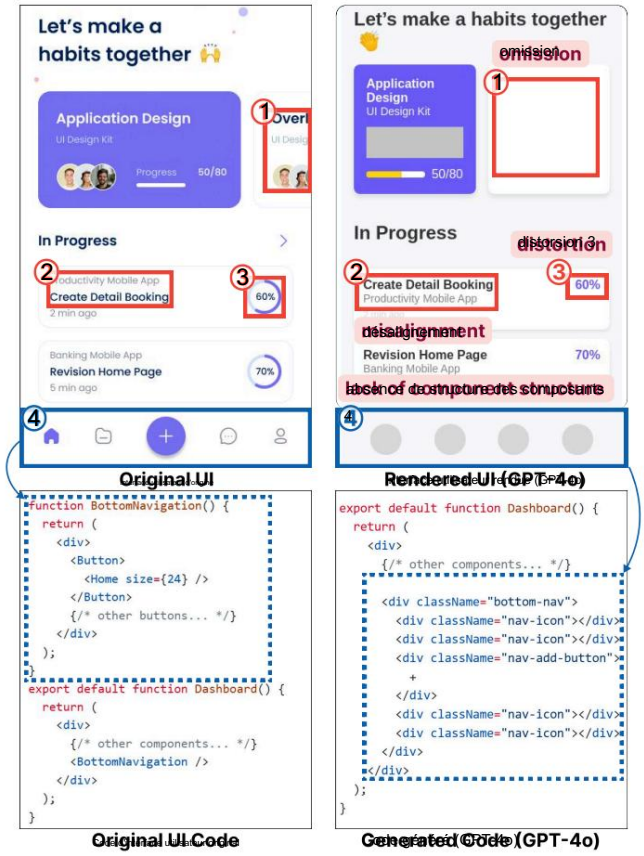


Figure 1 : Des exemples motivants sont fournis. Les cadres rouges 1, 2 et 3 mettent en évidence respectivement l'omission d'éléments, leur mauvaise disposition et leur distorsion dans l'interface utilisateur rendue. Le cadre bleu 4 indique que le code généré manque d'une structure bien définie et d' une sémantique fonctionnelle claire.

Lors de la reconstruction, les modèles linéaires multiniveaux (MLLM) sont sujets à des erreurs lors de la génération de code long, ce qui peut entraîner des éléments manquants et une désorganisation structurelle. De plus, les MLLM ont des difficultés à interpréter avec précision la structure hiérarchique des interfaces utilisateur complexes. Alors que certaines méthodes [31, 32] tentent d'inférer les structures d'interface utilisateur à partir de règles prédéfinies, l'évolution rapide de la conception d'interfaces réduit leur capacité d'adaptation aux nouveaux modèles. Les encadrés bleus de la figure 1 illustrent un exemple de code React généré. On observe qu'une identification correcte de la structure hiérarchique des composants améliore non seulement la modularité du code généré, mais aussi la réactivité des composants. Par exemple, les boutons de la barre de navigation inférieure peuvent être réutilisés efficacement tout en s'adaptant dynamiquement aux différentes tailles d'écran. De plus, les modèles de langages de modélisation hiérarchique traditionnels (MLLM) ne disposent pas de capacités d'analyse de programme et sont incapables de détecter les erreurs de rendu, ce qui entrave l'identification et la correction rapides de ces problèmes.

Pour relever ces défis, nous proposons DesignCoder, une nouvelle méthode de génération de code GUI prenant en compte la hiérarchie et s'auto-corrigeant. Pour améliorer la capacité de MLLM à reconnaître les structures hiérarchiques, nous proposons une chaîne de regroupement d'interface utilisateur qui intègre le traitement visuel et Le raisonnement par regroupement s'effectue selon un processus en chaîne. L' arbre des composants, construit par la chaîne de regroupement de l'interface utilisateur, représente les relations hiérarchiques et sémantiques des composants au sein d'une maquette d'interface. Cet arbre est enrichi d'informations de style extraites des métadonnées de la maquette. Pour générer un code structuré pour les composants et le style, nous proposons une stratégie de génération de code de type « diviser pour régner » basée sur l'arbre des composants. Après la génération initiale du code, nous introduisons une réparation de code autonome, prenant en compte la vision, afin d'identifier et de corriger les problèmes. Cette approche multi-étapes garantit que le code d'interface généré est fidèle à la conception originale, tant au niveau de la fidélité visuelle que de la sémantique fonctionnelle.

Pour évaluer l'efficacité de DesignCoder, nous avons collecté un ensemble de données de 300 maquettes haute fidélité auprès de la communauté Figma et d'une entreprise internet de premier plan. Sous le framework React Native, notre méthode surpasse la meilleure solution de référence, avec une amélioration de 37,63 %, 9,52 % et 12,82 % des métriques de similarité visuelle (MSE, CLIP, SSIM), tout en améliorant significativement la similarité de la structure du code, avec une augmentation de 30,19 %, 29,31 % et 24,67 % respectivement pour TreeBLEU, Container Match et Tree Edit Distance. De plus, nous avons mené une étude utilisateur auprès de développeurs professionnels afin d'évaluer la qualité et la praticité du code généré. Les résultats indiquent que DesignCoder surpasse significativement la solution de référence sur plusieurs aspects clés, notamment la disponibilité du code (4,52 contre 3,57), l'efficacité des modifications (4,20 contre 3,24), la lisibilité (4,75 contre 4,43) et la maintenabilité (4,32 contre 3,32). Ces résultats soulignent la capacité de DesignCoder à générer du code front-end de haute qualité, prêt pour la production, à l' échelle industrielle.

Nos contributions sont les suivantes :

- Nous proposons UI Grouping Chain, une approche innovante qui exploite une chaîne de pensée multimodale pour guider les modèles de langage multimodal (MLLM) dans la reconnaissance de la mise en page de l'interface utilisateur grâce au traitement visuel et au raisonnement par regroupement.
- Nous présentons DesignCoder, une nouvelle méthode basée sur les MLLM pour la génération de code à partir d'interfaces graphiques, intégrant la prise en compte de la hiérarchie et l'autocorrection. Cette méthode génère des composants structurés tout en exploitant les métadonnées de conception, garantissant ainsi des performances supérieures en termes de fidélité visuelle et de qualité du code.
- Nous menons une évaluation complète de DesignCoder, en le comparant aux méthodes de référence les plus récentes. De plus, nous réalisons une étude utilisateur, validant l'efficacité et la praticité de notre approche.

2. Approche. La figure

2 illustre le flux de travail global de notre méthode, DesignCoder , qui prend en entrée des maquettes d'interface utilisateur. DesignCoder vise à accélérer le développement et l'itération dans des scénarios industriels réels , où la cohérence visuelle, la maintenabilité du code et l' ergonomie sont essentielles. En exploitant les métadonnées de conception, notre méthode acquiert des informations permettant d'améliorer la qualité du code généré . DesignCoder introduit la chaîne de regroupement d'interface utilisateur (section 3.1), qui comprend deux étapes clés :
(1) Traitement visuel : cette étape segmente les captures d'écran d'interface utilisateur en sous-régions en fonction de la sémantique visuelle.
(2) Raisonnement de groupement : MLLM utilise une stratégie d'incitation en deux étapes pour analyser la sémantique des éléments au sein de chaque sous-région.
Il intègre ensuite ces sémantiques avec des invites visuelles pour guider le MLLM dans la construction d'un sous-arbre hiérarchique de composants d'interface utilisateur.

Machine Translated by Google

Dans l'étape de génération de code (section 3.2), nous adoptons une stratégie de type
diviser pour régner, où chaque sous-arbre est traité indépendamment.
Les métadonnées de conception sont utilisées pour générer les styles CSS et la mise en page.
Les styles sont intégrés dans une invite unifiée pour guider le MLLM
lors de la génération de code. Enfin, lors du raffinement auto-correctif du code
étape (section 3.3), notre méthode vérifie et corrige itérativement le
Le code initial généré est comparé visuellement pour garantir un rendu visuel.
cohérence et exactitude.

2.1 Chaîne de regroupement d'interface utilisateur

Interpréter les structures de couches dans les maquettes de conception et produire
Le regroupement précis des étudiants est une tâche complexe pour les titulaires d'un LLM. Pour y remédier,
nous décomposons la tâche de regroupement en trois plus faciles à gérer
sous-tâches, propose une chaîne de regroupement d'interface utilisateur (Figure 3) et intègre
des informations visuelles pour améliorer l'efficacité du LLM dans ces domaines
tâches. Les trois sous-tâches comprennent la division de l'interface utilisateur, l'extraction sémantique,
et le regroupement des composants.

L'interface utilisateur comprend de nombreux éléments, et la couche supérieure
Les informations contenues dans les maquettes de conception posent des défis aux MLLM.
traitement. Des recherches antérieures [26] suggèrent qu'une stratégie de division et de conquête est nécessaire.
Cette stratégie constitue une solution efficace à ce problème. En partitionnant le
L'interface utilisateur est divisée en plusieurs régions et traitée séparément.
La charge sur les MLLM peut être considérablement réduite. Cependant, sur mobile
Dans la conception d'interfaces utilisateur, les limites explicites sont souvent absentes et les régions d'interface sont…
principalement définie à partir de relations sémantiques. Cette contrainte
rend les algorithmes de segmentation simples [26] inefficaces pour partitionner avec précision
diverses interfaces utilisateur mobiles. Pour relever ce défi,
Nous proposons un outil de division basé sur LLM qui effectue la division de l'interface utilisateur
en tirant parti des relations sémantiques.

Nous extrayons d'abord les métadonnées du fichier de conception, notamment l'identifiant, le type,
et bbox, qui servent respectivement à identifier les éléments, à fournir des références sémantiques et à
déterminer la position des éléments.

Les informations sont ensuite organisées dans une liste de couches couches.
structurée. Ensuite, étant donné l'image d'entrée couches, nous utilisons le visuel
et la division en plusieurs éléments
sous-listes des couches d'invite de traitement Ces listes
d'éléments définissent la position de chaque région
sous-couches
et dimensions. L'interface utilisateur entière est ensuite divisée en plusieurs sous-couches en fonction de ces éléments.
La segmentation se poursuit et le traitement passe à l'étape suivante.
Le processus peut être exprimé comme suit :

$$(\text{couche}) \quad , \text{boîte de réception} \quad , \text{taper} \quad = 1 \tag{1}$$

$$= \quad \text{diviser}(\ , \text{couches}) \tag{2}$$

Pour améliorer la fiabilité de la segmentation de l'interface utilisateur générée par LLM,
La validation et le post-traitement sont essentiels. Nous avons défini trois points clés.
Principes : (1) chaque élément appartient à une seule région, à l'exception de
Les grands conteneurs et les arrière-plans sont traités séparément ;
(2) les régions doivent être mutuellement exclusives et sans chevauchement ; (3) le
Le nombre de régions devrait rester stable, avec des partitions en dehors de
Plage de 3 à 10 nécessitant une resegmentation. Comme indiqué dans l'algorithme
1, en intégrant un LLM avec un algorithme de correction de division,
Notre méthode capture efficacement les informations sémantiques et spatiales,
garantir une segmentation de haute précision.

Le regroupement des éléments d'interface utilisateur dépend à la fois des relations spatiales et sémantiques .
Pour intégrer efficacement les informations sémantiques, il est nécessaire de prendre en compte ces dernières.
Intégrée aux couches d'interface utilisateur, nous avons développé une extraction sémantique sur mesure
rapide pour les calques d'interface utilisateur. Pour chaque sous-région segmentée, nous

Algorithme 1 : Vérification et post-traitement

Exigences : Entrée requise, Vérifier :
Divisions corrigées ˆ, où chaque couche est affectée à
exactement une division.
si | | < 3 | | > 10 alors
Rétablir la version précédente (,)
pour tout faire
Calculer la boîte englobante bbox() en fusionnant les boîtes englobantes de
pour toutes les couches faire
← { | bbox( ) ∩ bbox( ) ≠ }
si = alors
Insérer dans la division sinon dont le centre est le plus proche de
Fusionnez toutes les divisions en une seule division et
insérer dans
Mettre à jour la boîte de base de la ou des divisions concernées
pour toutes les paires de divisions distinctes (,) faire
si bbox() ∩ bbox() ≠ alors
Fusionner et
Retour aux divisions corrigées ˆ

recadrer l'image d'interface utilisateur correspondante, extraire les métadonnées de conception pertinentes
(c.-à-d., type, bbox), et leur fournir l' entrée demandée à un MLLM. Le comme
modèle produit ensuite une liste d'éléments contenant des annotations sémantiques. Par
exemple, pour le texte dans une recherche
Dans la barre, la description générée pourrait être : « Un texte d'espace réservé affichant "Rechercher
des combinaisons de salades de fruits", ce qui suggère la recherche souhaitée ».
« Portée ou exemple de cas d'utilisation pour la barre de recherche. » Ce processus donne un résultat
Liste sémantique de couches . Pour affiner les résultats, nous appliquons une correction de post-traitement et utilisons
des techniques d'amélioration visuelle.
par exemple, annoter les éléments difficiles à identifier dans l'image,
garantir une extraction complète des éléments d'interface utilisateur.

Nous introduisons une invite de regroupement structurée qui organise les éléments en un arbre de
composants hiérarchique, en utilisant comme entrées la sémantique des listes sémantiques de couches
et les sous-images. Pour garantir
_
Pour garantir la fiabilité, nous mettons en œuvre une correction post-traitement afin d'éviter
Les nœuds feuilles sont générés aléatoirement et les composants qui se chevauchent sont
éliminés . Cette approche structurée transforme une liste d'éléments aplatie.
dans une hiérarchie bien définie, servant de base au front-end
Génération de code. Ce processus peut être exprimé comme suit :

$$\text{sémantique} = \text{sémantique}(\text{sous-couches, sous-image}) \tag{3}$$

$$\text{sous-arbre} = \text{groupe}(\text{sémantique, sous-image}) \tag{4}$$

2.2 Génération de code prenant en compte la hiérarchie

Une implémentation frontale déclarative complète se compose de deux
Étapes clés : code du composant et code CSS. Pour le composant
Pour la génération de code, nous utilisons l'arbre de composants hiérarchiques de l'interface utilisateur.
Généré par une chaîne de regroupement d'interface utilisateur en entrée. Notre
invite de code comprend quatre étapes : attribution des rôles, description de la tâche et détails
exigence et exemple de résultat. Le LLM est désigné comme un React
Un expert a reçu pour instruction de générer des composants React en respectant scrupuleusement
l'arborescence des composants. Il attribue des noms de sous-régions aux composants.

Machine Translated by Google

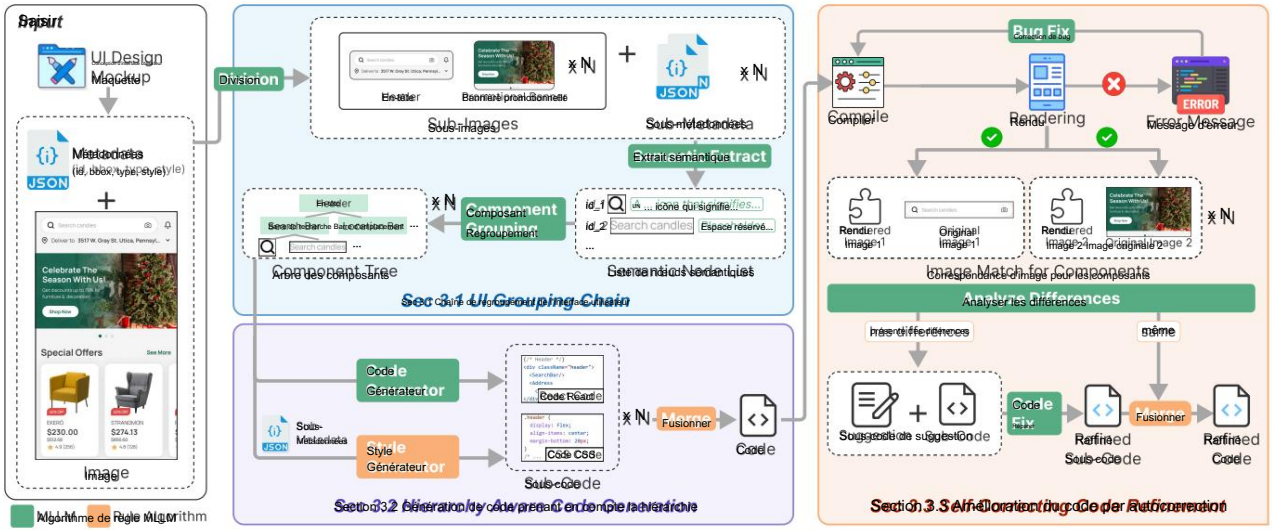


Figure 2 : Vue d'ensemble de DesignCoder.

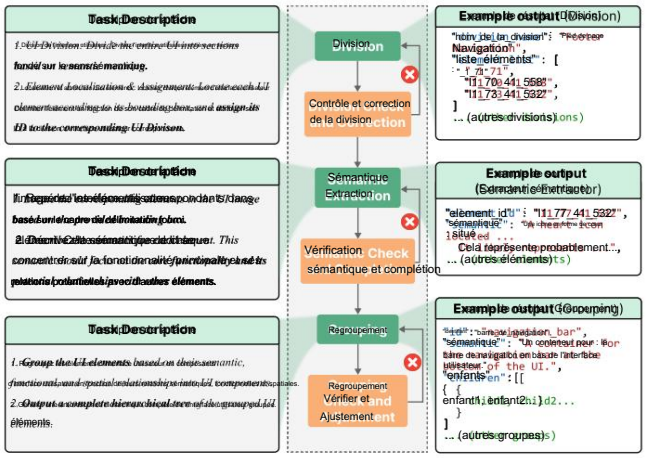


Figure 3 : L'invite utilisée pour construire la chaîne de regroupement de l'interface utilisateur.

des identifiants, assurant une association transparente avec les définitions de style et facilitant l'assemblage de la page entière. Tirant parti de sa compréhension de la sémantique des composants et des comportements d'interaction, le LLM sélectionne les balises de composants appropriées et leur attribue des gestionnaires d'événements. tout en laissant le contenu de l'événement non spécifié, facilitant ainsi le développement ultérieur de l'application. Par exemple, dans le cas des vues défilables, si la conception définit explicitement un comportement de défilement, notre modèle peut Sélectionner intelligemment le composant <ScrollView> plutôt que d'appliquer simplement overflow: scroll. Cette approche améliore le code. Lisibilité et maintenabilité. Ce processus peut être exprimé comme suit :

$$\text{code} = \text{code}(\text{sous_arbre}, \text{sous_image}) \quad (5)$$

CSS fournit des styles visuels tels que les couleurs, la mise en page, les bordures et espacement. Afin de maintenir une cohérence visuelle entre les éléments générés À partir du code et de l'interface utilisateur d'origine, nous utilisons des métadonnées de conception et des données structurées. Informations sur les composants. Pour la génération de la mise en page, nous parcourons l' arbre hiérarchique (couches) de bas en haut. Chaque nœud feuille correspond à un élément et conserve ses attributs bbox d'origine, tandis que non-feuille Les nœuds agrègent les données de boîtes englobantes des nœuds enfants pour déterminer taille et positionnement relatif. Cette agrégation permet de garantir un alignement précis et une cohérence visuelle. Attributs du texte (tels que Le style, la couleur, la police et l'épaisseur des caractères sont systématiquement extraits pour calculer les tailles relatives, l'espacement et les hauteurs de ligne. De plus, le remplissage, Les bordures, les ombres et les styles d'angle sont générés en fonction de les métadonnées de conception. Enfin, le code structuré du front-end est organisé en sous-régions, reflétant fidèlement les propriétés de mise en page de l'interface utilisateur.

assurer une réactivité optimale sur différents écrans.

2.3 Amélioration automatique du code

Étant donné que les LLM sont générés de manière autorégressive, une fois qu'une erreur se produit si cela se produit pendant le processus de génération, les LLM continueront à La génération de contenu à partir d'erreurs entraîne la propagation de ces erreurs. Le code généré par les MLLM peut contenir divers éléments. erreurs, notamment des éléments manquants, des distorsions de style, des incohérences de mise en page , des erreurs de syntaxe et des anomalies fonctionnelles.

Les études [16, 34] se concentrent sur la garantie d'une analyse et d'une exécution correctes du code ; une analyse correcte ne garantit pas nécessairement que… Le code généré est visuellement et fonctionnellement précis. Par exemple, Le code généré peut omettre l'icône de recherche dans la zone de recherche. une barre ou un résultat désorganisé dans la mise en page des éléments, affectant en fin de compte l'ergonomie. Pour remédier à ces problèmes, nous proposons une solution auto-correctrice. stratégie basée sur l'arbre de composants généré, qui évalue systématiquement la cohérence visuelle et la réactivité de la mise en page. code généré. Nous utilisons ensuite une approche de type diviser pour régner. Effectuer des corrections de code précises et fusionner le code affiné.

Plus précisément, nous utilisons Appium, un outil de test d'automatisation d'interface utilisateur, pour Capture d'écran de la page rendue et extraction des attributs des composants. y compris les boîtes englobantes, les types d'éléments, le contenu textuel et les structures hiérarchiques. Ensuite, nous faisons correspondre les composants rendus à l'aide de Nous extrayons ensuite les identifiants des composants à partir de l'arborescence des composants. Puis, nous extrayons l'image.

Machine Translated by Google

segments basés sur la boîte englobante de chaque composant, obtenant ainsi les image du composant d'origine et image du composant rendu
.

Nous utilisons ensuite une invite d'analyse visuelle, une analyse, à méticuleux-
comparons attentivement ces images, en nous concentrant sur l'identification de tout élément Erreurs de mise en page ou de style dans le code généré :

$$= \text{analyse}(,) \quad (6)$$

où représente la suggestion de réparation pour le -ième composant, avec correspondant à l'image originale et à l'image rendue image.

Par la suite, la suggestion de réparation générée est combinée avec l'extrait de code du composant associé pour former une structure Réparation du code invite de réparation :

$$= \{\text{“component_code_i”}, \} \text{ réparation} \quad (7)$$

Cette invite guide le MLLM dans l'exécution de réparations de code ciblées . Notamment, puisque chaque composant peut être réparé indépendamment, le processus peut être parallélisé, ce qui améliore considérablement l'efficacité de Réparation du code. Enfin, nous fusionnons les extraits de code en fonction de Ordre de l'arborescence des composants pour produire la page finale rendue. Figure La figure 4 présente trois exemples, dont celui généré initialement. Le code et son code raffiné correspondant. Nous avons identifié trois cas typiques. types d'erreurs, mis en évidence dans la figure 4 : cases rouges Les cases bleues indiquent les éléments mal alignés, les cases jaunes les éléments déformés et les cases jaunes les éléments manquants. En comparant En comparant les résultats rendus avec l'interface utilisateur d'origine, nous avons constaté que MLLM détecte efficacement les erreurs potentielles et localise avec précision les segments de code correspondants. Par exemple, dans le cas 1, le cadre bleu marque un fond bleu généré incorrectement, tandis que le cadre rouge met en évidence les éléments mal alignés. Notre méthode a détecté avec succès Ces erreurs ont été corrigées et corrigées. Dans le cas 2, notre Cette approche a non seulement permis d'identifier les éléments qui se chevauchent (marqués dans le (encadré rouge) mais a également ajusté les coins arrondis d'un arrière-plan image (marquée dans le cadre bleu), démontrant sa capacité à réaliser des corrections de code précises. Cette amélioration découle de notre stratégie de décomposition de la tâche de réparation du code en fonction de l' arbre des composants généré, employant une approche de type diviser pour régner guide MLLM dans le processus de correction. Ces résultats suggèrent que améliorer la compréhension des erreurs dans le code généré par les MLLM est un Facteur clé pour améliorer la qualité de la génération de code.

3 Évaluation

Pour évaluer de manière exhaustive les performances de DesignCoder, nous Ils ont conçu une série d'expériences pour aborder les trois points clés suivants questions de recherche (QR) :

- RQ1 : Efficacité. Comment DesignCoder se comporte-t-il ? en termes de cohérence visuelle et de similarité structurelle par rapport aux méthodes de référence ?
- RQ2 : Étude d'ablation. Quel est le rôle de chaque élément clé ? composante de la performance globale de DesignCoder ?
- RQ3 : Étude utilisateur et étude de cas. Quelle est la perception ? qualité du code généré par DesignCoder à partir d'un utilisateur perspective?

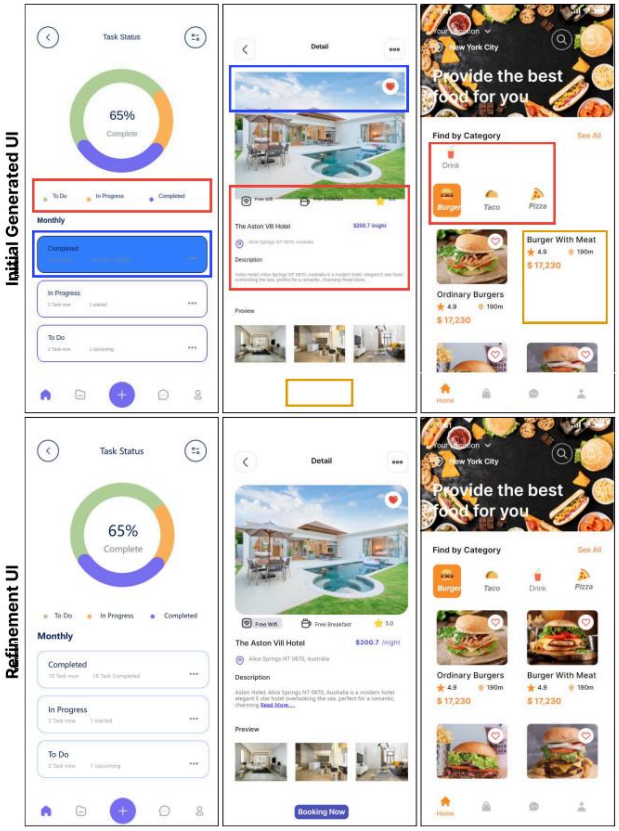


Figure 4 : Exemples de résultats de rendu obtenus à partir des deux Le code initialement généré et le code de raffinement. Le rouge Les encadrés mettent en évidence les éléments mal alignés, les encadrés bleus indiquent distorsions visuelles, et les cases jaunes indiquent les éléments manquants .

3.1 Ensemble de données

Nous avons constitué un ensemble de données contenant 300 interfaces utilisateur d'applications mobiles. Des maquettes de conception, réparties en cinq catégories : divertissement, éducation, santé, shopping et voyages. Les données proviennent de la communauté Figma et une grande entreprise internet utilisant Sketch logiciel. Plus précisément, il comprend 250 maquettes issues de la communauté Figma et 50 issues de la source entreprise. Les fichiers Sketch étaient importées dans Figma, nous avons utilisé l'interface de ligne de commande de Figma pour extraire ressources d'images et métadonnées de conception standardisées. Surtout, puisque Les structures hiérarchiques des maquettes de conception ne correspondent pas directement aux structures de code ; notre méthode n'en dépend donc pas. Structures de couches originales.

3.2 Dispositif expérimental

3.2.1 Méthodes de référence. Dans cette étude, nos méthodes de référence peuvent être classées en deux catégories : les méthodes spécialisées pour l'interface utilisateur mobile et les méthodes de référence pour l'interface utilisateur mobile. génération et MLLM à usage général. Les méthodes spécialisées inclure:

- Prototype2Code [30] améliore la structure hiérarchique de conception de maquettes en utilisant le regroupement d'éléments fragmentés [8] et des techniques de regroupement sémantique [32]. Il utilise des règles élaborées manuellement pour générer un arbre de mise en page et reconnaître les éléments.

Machine Translated by Google

DeclarUI [34] utilise des modèles pour construire le squelette HTML, puis exploite un modèle de langage étendu pour générer le code CSS. Il intègre un modèle de détection d'éléments non supervisé [20] et un modèle de segmentation [17] pour générer la description fonctionnelle des composants d'interface utilisateur. Il introduit ensuite des graphes de transition de page pour construire la logique interactive, qui sert à interroger les modèles de langage pour générer des interfaces utilisateur d'applications mobiles avec une logique de saut. Nous utilisons sa version GPT-4o comme référence.

Les MLLM à vocation généraliste comprennent :

- GPT-4o, développé par OpenAI, présente des capacités exceptionnelles de compréhension d'images et peut générer du code à partir d'images. Conformément aux meilleures pratiques de recherche existantes sur la conversion d'images en code [25, 28, 30], nous avons conçu avec soin les instructions pour GPT-4o (plus précisément, GPT-4o-2024-11-20).
- Claude-3.5, développé par Anthropic, est un modèle multimodal de grande taille qui a démontré des performances remarquables sur de nombreux benchmarks MLLM. Nous utilisons Claude-3.5-Sonnet- 20241022 comme référence.
- LLaVA-v1.5-7B [19] est un modèle multimodal de grande taille entraîné de bout en bout qui permet une compréhension générale de la vision et du langage en intégrant un encodeur de vision avec un modèle de langage de grande taille. Nous utilisons les mêmes invites que GPT-4o pour générer du code d'interface utilisateur à partir d'images.

3.2.2 Métriques d'évaluation. Notre évaluation utilise un ensemble de métriques pour évaluer les performances de DesignCoder, englobant la similarité visuelle, la similarité structurelle et la facilité d'utilisation pratique.

Pour évaluer la similarité visuelle, nous utilisons des captures d'écran d'interface utilisateur extraites de maquettes d'interface utilisateur comme référence et introduisons trois métriques :

- L'erreur quadratique moyenne (MSE) est une métrique courante pour évaluer la similarité d'images en mesurant les différences pixel par pixel entre deux images. Il calcule la différence quadratique moyenne entre les valeurs de pixels correspondantes, où les valeurs les plus basses indiquent une plus grande similarité.
- Score CLIP [25]. Cette métrique quantifie la similarité sémantique entre les interfaces utilisateur générées et originales, définie par cos clip(image), clip(image) , qui mesure la similarité cosinus entre les représentations des caractéristiques CLIP des images.
- L'indice de similarité structurelle (SSIM) [27]. Il évalue la précision de la mise en page et de la composition, en se concentrant sur les similarités d'agencement spatial entre les interfaces utilisateur générées et originales. Une limite du SSIM est qu'il n'évalue que la similarité structurelle au niveau visuel, sans évaluer la similarité structurelle et sémantique au niveau du code.

Par conséquent, afin d'évaluer la similarité structurelle du code généré, nous avons adopté une combinaison d' implémentation de code automatisée et manuelle. Pour obtenir la vérité de terrain du code d'interface utilisateur, nous introduisons Codefun [1], une plateforme de génération de code de niveau entreprise, qui convertit des fichiers de conception bien structurés en code front-end.

Par la suite, deux auteurs ont examiné manuellement l' arbre des composants généré, en se concentrant sur la hiérarchie et les étiquettes des composants. En cas d'erreurs de hiérarchie structurelle, chaque auteur a corrigé les problèmes indépendamment avant de parvenir à un consensus par la discussion.

De plus, les auteurs ont fait référence à Material Design 3 pour corriger les types de composants incorrects. Par exemple, un bouton texte étiqueté comme Un élément de type « texte » a été corrigé en un élément de type « bouton ». Ensuite, nous introduisons trois indicateurs pour comparer les structures hiérarchiques :

TreeBLEU [13] est utilisé pour évaluer la similarité entre l'arbre DOM du HTML généré et le DOM de référence, en excluant les nœuds terminaux contenant des attributs de balise (par exemple, `content` et `styles`). Il est défini comme la proportion de sous-arbres de hauteur 1 dans l'arbre donné qui correspondent à des sous-arbres de l' arbre de référence. TreeBLEU fournit une mesure globale de la similarité des arbres DOM HTML. Nous avons adapté cette métrique pour améliorer son applicabilité à l'évaluation de code mobile. Elle peut être formulée comme suit :

$$\text{ArbreBLEU} = \frac{|\ (\ ) \cap (\ )|\ |\ (\ )|}{\text{et}} \text{ où}, \tag{8}$$

désignent respectivement l'arbre donné et l'arbre de référence . • La distance d'édition d'arbres (TED) est une métrique permettant de mesurer la similarité entre arbres en fonction du nombre minimal d'opérations d'« édition » nécessaires pour transformer un arbre en un autre arbre isomorphe.

$$\text{TED}(\ 1,\ 2) = \min_{(\ 1,\ldots,\ )\ P(\ 1,\ 2\ )} \sum_{=1} (\ ) \tag{9}$$

P(1, 2) désigne l'ensemble des chemins d'édition possibles entre et 2. Un TED 1 plus faible indique de meilleures performances. Par conséquent, s'il n'y a pas de nœuds correspondants dans le résultat prédit, nous attribuons au TED la valeur du nombre d'arêtes dans l'arbre de vérité terrain. • La correspondance de conteneurs (CM) [29] se concentre sur le regroupement des éléments plutôt que sur leur structure spécifique. Cette métrique est particulièrement pertinente pour certaines tâches en aval, telles que la segmentation d'écran, où l'interface est divisée en régions distinctes. La métrique est calculée en faisant la moyenne des scores d'intersection sur union (IoU) de chaque conteneur (par exemple, les nœuds intermédiaires) par rapport aux annotations de vérité terrain. Le score de correspondance de conteneurs varie de 0 à 1, où CM = 1 indique un alignement de regroupement parfait. Pour les structures arborescentes sans nœuds correspondants, nous attribuons un score de 0.

3.2.3 Détails d'implémentation. Conformément à DeclarUI, nous avons sélectionné GPT-4o comme modèle de base. Dans nos expériences, tout le code généré a été implémenté à l'aide de frameworks de développement mobile populaires, notamment React Native [21]. Nous avons automatisé l' intégration du code généré dans un projet initial sous Android Studio grâce à des scripts Python afin de minimiser l'intervention humaine et les biais. Pour le code n'ayant pas pu être compilé, nous avons attribué la valeur zéro à toutes ses métriques d'évaluation.

3.3 RQ1 : Métrique visuelle

d'efficacité. Le tableau 1 indique que DesignCoder surpasse toutes les méthodes de référence en ce qui concerne les métriques visuelles. Comparée à la méthode de référence de pointe Prototype2Code, notre approche obtient des améliorations de 37,64 %, 9,52 % et 12,82 % pour les scores MSE, CLIP et SSIM sur l'ensemble de données Figma, et de 24,31 %, 4,60 % et 11,69 % sur l' ensemble de données de l'entreprise. Les scores SSIM et CLIP indiquent que, tant en termes de structure de l'image que de sémantique, le code généré par DesignCoder correspond étroitement aux jugements perceptifs humains. Pour le score MSE, une valeur plus faible indique une meilleure cohérence visuelle, suggérant que DesignCoder génère un code d'interface utilisateu

Machine Translated by Google

reproduit le dessin original. En effet, notre méthode capture
les relations hiérarchiques des composants d'interface utilisateur et les intègre
Métadonnées de conception, permettant une meilleure fidélité visuelle. Comparaison
à DeclarUI, qui repose uniquement sur la saisie d'images, notre méthode
le surpasse en utilisant le même modèle de base (GPT-4o).
En effet, bien que l'utilisation de modèles CV puisse améliorer les MLLM,
compréhension des images d'interface utilisateur, les relations hiérarchiques complexes
dans l'interface utilisateur et les variations à grande échelle des composants d'interface utilisateur (tels que
Les images d'arrière-plan et les icônes posent des défis importants à la
capacité de généralisation des modèles de vision par ordinateur. Notre méthode exploite l'interface utilisateur.
chaîne de pensée multimodale pour améliorer les capacités des MLLM en matière d'interface utilisateur
compréhension, permettant une compréhension plus approfondie de la sémantique
relations entre les composants de l'interface utilisateur. De plus, nous introduisons
un mécanisme d'amélioration du code auto-correcteur, permettant au LLM
détecter les problèmes subtils et les corriger de manière autonome, par conséquent
obtenir une plus grande cohérence visuelle avec l'interface utilisateur d'origine.

Métrique structurelle. Nous avons constaté que les métriques visuelles seules
ne peut pas refléter pleinement la qualité du code généré. En pratique , les
développeurs front-end accordent une plus grande importance à la question de savoir si
Le code respecte les bonnes normes structurelles. Un code bien structuré est
généralement plus faciles à entretenir et à modifier. Par conséquent, nous
introduction de TreeBLEU, Tree Edit Distance et Container Match à
évaluer la structure du code.

Les résultats du tableau 2 montrent que, par rapport à la meilleure référence,
Notre méthode améliore TreeBLEU, Container Match et Tree Edit
Distance sur l'ensemble de données Figma de 30,19 %, 29,31 % et 29,47 %, sur
l'ensemble de données de l'entreprise de 17,78 %, 22,45 % et 26,33 %, respectivement.
Cela indique que notre approche permet de mieux appréhender les types de
composants communs et les relations hiérarchiques. Il convient de noter que
Bien que DeclarUI prenne en compte la logique de navigation des pages, il néglige
les relations entre les composants d'une même page, ce qui affecte le résultat généré
structure du code. Plus important encore, dans l'ensemble de données de l'entreprise, qui
Grâce à des relations entre composants plus complexes, la structure de code générée par
DesignCoder est également mieux organisée. Ceci démontre une fois de plus la forte
capacité de généralisation de DesignCoder et
adaptabilité, grâce aux données d'entraînement exhaustives et aux paramètres à grande
échelle des MLLM.

Tableau 1 : Comparaison des métriques de similarité visuelle entre différentes méthodes (la meilleure est marquée en gras).

	Jeu de données Figma			Ensemble de données de l'entreprise		
Modèle	MSE↓	CLIP↑	SSIM↑	MSE↓	CLIP↑	SSIM↑
LLava-v1.5-7B	91,49	0,58	0,58	85,28	0,57	0,56
GPT-4o	70,59	0,69	0,61	72,13	0,61	0,69
Claude-3.5	82,47	0,67	0,52	83,62	0,62	0,55
DeclarUI	52,83	0,79	0,83	53,76	0,75	0,70
Prototype2Code	36.32	CodeFun	0,84	0,78	28.18	0,87	0,77
DesignCoder	28.45	0,93	0,71	58,55	0,91	0,88	0,57
	22,65	0,92		21,33	0,91		0,86

3.4 RQ2 : Étude d'ablation

Nous avons mené une étude d'ablation pour étudier l'efficacité de
les composants clés de DesignCoder. Les résultats de ces ablations
Les études sont présentées dans le tableau 3. Dans la première expérience d'ablation,
Nous avons supprimé le module UI Grouping Chain. À la place, nous avons directement…
Les MLLM ont été invités à générer l'arborescence des composants à partir de captures d'écran de
l'interface utilisateur et d'une liste de calques. Suppression de la chaîne de regroupement de l'interface utilisateur
Le module a entraîné une baisse significative des scores de similarité visuelle. Plus précisément,
le score CLIP est passé de 0,92 à 0,84 et le SSIM a diminué.
de 0,88 à 0,79, et l'erreur quadratique moyenne (MSE) est passée de 22,43 à 38,92.
La baisse des indicateurs visuels indique directement une capacité réduite de la
MLLM pour générer des arbres de composants structurellement corrects. Ce
résultat souligne la difficulté de permettre à MLLM de traiter des modèles complexes.
Pages d'interface utilisateur directement. Notre méthode aide MLLM à décomposer les problèmes complexes
décomposer les tâches en sous-tâches plus simples en décomposant les images d'interface
utilisateur en plusieurs sous-images, réduisant ainsi la charge cognitive. De plus,
Les scores de similarité de la structure du code ont également chuté de manière significative, avec
TreeBLEU diminue de 42,4 % et Container Match de 43,8 %.
Ce déclin est probablement dû à l'absence de sémantique au niveau des composants.
Le raisonnement dans MLLM. Les relations sémantiques améliorent la capacité de MLLM
à inférer la logique de regroupement, permettant la reconnaissance de structures complexes.
structures hiérarchiques.

Dans la seconde expérience d'ablation, nous avons supprimé le module d'amélioration du
code. Comme le montre le tableau 3, la structure visuelle et la structure du code ont été modifiées.
Les scores de similarité ont diminué. Nous avons observé que MLLM peut identifier certains
problèmes de style détaillés, tels que la taille de la police, les éléments manquants ou mal agencés.
éléments et distorsion des couleurs, et les corriger efficacement dans le
Le code. De plus, une comparaison visuelle peut révéler la structure du code.
Des problèmes, tels que des conteneurs manquants, entraînent des erreurs relatives.
positionnement. Notre méthode est capable de détecter ces problèmes et
apporter des corrections en temps opportun.

3.5 RQ3 : Étude utilisateur et étude de cas

3.5.1 Étude utilisateur. Dans le cadre d'un développement industriel à grande échelle, l'ensemble
La qualité du code est un critère essentiel pour évaluer les performances.
des méthodes de génération automatique de code [8, 30, 34]. À cette fin, nous
nous avons mené une étude auprès des utilisateurs, en invitant des développeurs expérimentés à évaluer
le code généré et participer à des entretiens pour en discuter
justification de leurs évaluations.

Procédure. Nous avons recruté cinq participants âgés de plus de cinq ans.
d'expérience en développement front-end avec React Native
cadre. Ces participants peuvent être considérés comme des développeurs front-end senior.
des développeurs capables d'évaluer la qualité du code généré.
Pour mener notre étude, nous avons sélectionné au hasard 15 prototypes de conception d'interface utilisateur.
représentant trois catégories typiques de conception d'interface utilisateur : voyage,
communication et achats. Parmi celles-ci, 10 prototypes ont été sélectionnés.
Nous avons utilisé DeclarUI, équipé de GPT-4o, comme référence .

à des fins de comparaison. Chaque participant a examiné le code indépendamment.
généré à la fois par DesignCoder et par la ligne de base. Conformément à [8],
Nous avons évalué le processus de modification sur la base de deux indicateurs clés :
Disponibilité du code et délai de modification. Après avoir effectué les modifications, les
participants ont évalué la lisibilité et la maintenabilité.
du code à l'aide d'une échelle de Likert à cinq points. La disponibilité du code était
Le suivi s'effectuait via les journaux Git, tandis que le temps de modification mesurait la
durée nécessaire pour adapter le code aux normes de production.
a été catégorisée en intervalles de cinq minutes, un score de 5 indiquant une
réalisation en cinq minutes et un score de 1 indiquant
L'exécution a duré plus de 20 minutes. Tous les participants ont réalisé le
Évaluations réalisées avec Android Studio.

Machine Translated by Google

Tableau 2 : Comparaison des métriques de similarité structurelle entre différentes méthodes (la meilleure est marquée en gras).

Modèle	Jeu de données Figma			Ensemble de données de l'entreprise		
	TreeBLEU↑	Correspondance du conteneur↑	Distance d'édition de l'arbre↓	TreeBLEU↑	Correspondance du conteneur↑	Distance d'édition de l'arbre↓
LLava-v1.5-7B	0,19	0,25	49.11	0,11	0,18	42,54
GPT-4o	0,24	0,31	49,85	0,22	0,30	32,77
Claude-3.5	0,24	0,33	41,49	0,19	0,27	35,62
DeclarUI	0,32	0,41	43.12	0,28	0,34	31.10
Prototype2Code	0,53	0,58	40,00	0,45	0,49	28,50
CodeFun	0,52	0,61	35,63	0,41	0,52	30,45
DesignCoder	0,69	0,75	28.21	0,53	0,60	22,86

Tableau 3 : Métriques de performance complètes pour l'étude d'ablation (deux ensembles de données fusionnés, le meilleur étant indiqué en gras).

Méthode	MSE↓	CLIP↑	SSIM↑	TreeBLEU↑	Correspondance de conteneur↑	Distance d'édition d'arbre↓
DesignCoder sans	22.43	0,92	0,88	0,66	0,73	27,32
regroupement : 38,92 ; sans		0,84	0,79	0,38	0,41	36,57
raffinement : 33,64		0,88	0,85	0,55	0,64	29.44

Résultats. Le tableau 4 présente les scores moyens, les valeurs p étant calculées à l'aide du test U de Mann-Whitney. Globalement, DesignCoder surpasse la solution de référence pour quatre indicateurs d'évaluation de la qualité du code : disponibilité du code, délai de modification, lisibilité et maintenabilité, avec des scores moyens respectifs de 4,52, 4,20, 4,75 et 4,32. Notamment, Notre approche présente des différences significatives par rapport à DeclarUI sur tous les points. quatre indicateurs. Des scores de maintenabilité et de lisibilité plus élevés indiquent que le code généré par DesignCoder est plus facile à maintenir en environnement de production. Les participants ont unanimement convenu que Le code généré par DesignCoder présente une meilleure conception réactive et sa modularité. Ceci est attribué à la hiérarchie bien structurée. parmi les composants et la capacité de DesignCoder à identifier correctement relations entre eux. Le participant P1 a commenté : « Comparé par rapport aux LLM que j'ai utilisés auparavant, la structure (du code généré) « est beaucoup plus proche de ce que j'écris dans mes projets. » De plus, DesignCoder réduit considérablement le temps de modification du code (DesignCoder : 4,20 contre 3,24 (valeur de référence), ce qui suggère que le code généré par DesignCoder il est plus facile de s'adapter aux normes de production. Le participant P3 a noté : « La décomposition en composants de ce code (généré par DesignCoder) est « Mieux, ce qui me permet de le réutiliser à plusieurs endroits. » Cet avantage est Cela se reflète également dans la disponibilité du code (DesignCoder : 4,52 contre Référence : 3.57), où les modifications apportées à un composant généré par Design-Coder se propagent automatiquement à toutes les instances où il est utilisé. Enfin, nous avons demandé aux participants s'ils envisageraient d'utiliser DesignCoder. La plupart des réponses ont été positives, avec des commentaires tels que : « Je serais prêt à essayer. Cela pourrait me faire gagner beaucoup de temps pour les interfaces utilisateur complexes. plans.

3.5.2 Étude de cas. Pour démontrer la supériorité de DesignCoder, Nous fournissons des exemples précis dans les figures 5 et 6. Nous avons sélectionné deux méthodes de base, DeclarUI [34] et Prototype2Code [30], pour comparaison. DeclarUI représente l'approche SOTA qui génère Le code frontend provient de captures d'écran de l'interface utilisateur, tandis que Prototype2Code est le Une approche de pointe qui génère du code frontend à partir de maquettes d'interface utilisateur. L'analyse révèle que DesignCoder surpasse ces solutions de référence dans trois aspects clés : fidélité visuelle, exhaustivité sémantique des composants, et une compatibilité réactive. Nous illustrons ces aspects à l'aide de Fichiers de conception provenant de la communauté Figma et de l'entreprise.

La figure 5 illustre que DesignCoder offre une qualité visuelle supérieure fidélité grâce à une reconstruction précise des agencements et des styles des composants. Les cadres rouges mettent en évidence des incohérences visuelles entre DeclarUI et Prototype2Code . Plus précisément, DeclarUI présente des composants manquants. (par exemple, les icônes et les images de fond) et le texte mal aligné. Bien que Pro- totype2Code conserve un niveau de précision visuelle relativement élevé, il Il souffre encore de textes mal placés et d'éléments d'arrière-plan manquants.

En termes d'intégrité des composants, notre approche identifie efficacement les modèles de conception et réutilise les composants identiques, garantissant ainsi Un code propre et maintenable. Les cases bleues indiquent un code bien structuré. composants du code, tandis que les cadres jaunes mettent en évidence les éléments incorrects implémentations qui reposent uniquement sur des éléments « ». Nous observons DeclarUI détecte correctement les composants « Carte » autonomes ; cependant , il peine à reconnaître les sections et les éléments de liste composés. de plusieurs composants, en raison des limitations de sa vision par ordinateur Prototype2Code, en revanche, ne parvient pas à reconnaître entièrement les composants, car il utilise une approche basée sur des règles pour l'arbre DOM. génération, négligeant la sémantique des éléments. En revanche, notre méthode exploite les puissantes capacités de compréhension sémantique de MLLM. Grâce à un processus de génération en plusieurs étapes, notre approche garantit la mise en œuvre correcte de composants complexes.

Un autre avantage clé de notre méthode est son support robuste pour La conception adaptative, comme illustré à la figure 6, assure un rendu optimal quelle que soit la taille de la fenêtre en déterminant avec précision… hiérarchie de code, permettant un calcul précis des tailles relatives et positions pour une adaptation fluide entre appareils. Tandis que DeclarUI démontre un certain degré de réactivité, son style généré Le code est incomplet, ce qui entraîne des résultats de rendu sous-optimaux. Prototype2Code , malgré une fidélité visuelle élevée à l'origine La taille de la page ne prend pas en charge l'adaptation multi-écrans. En revanche, Notre approche établit un équilibre entre rendu haute fidélité et un comportement réactif sur différentes tailles d'écran, ce qui en fait un Solution concrètement viable pour les applications industrielles.

4 Menaces à la validité

Deux menaces principales pèsent sur la validité de notre approche. La menace qui pèse sur la validité interne réside dans la fiabilité des indicateurs d'évaluation.

Machine Translated by Google

Tableau 4 : Évaluations des participants concernant DesignCoder par rapport à la ligne de base selon différentes métriques.

Méthode	Disponibilité du code, délai de modification, lisibilité, maintenabilité			
DeclarUI	3,57	3.24	4.43	3,32
DesignCoder	4,52	4.20	4,75	4.32
Valeur p	2,10E-6	4,57E-5	4,96E-2	1,51E-6

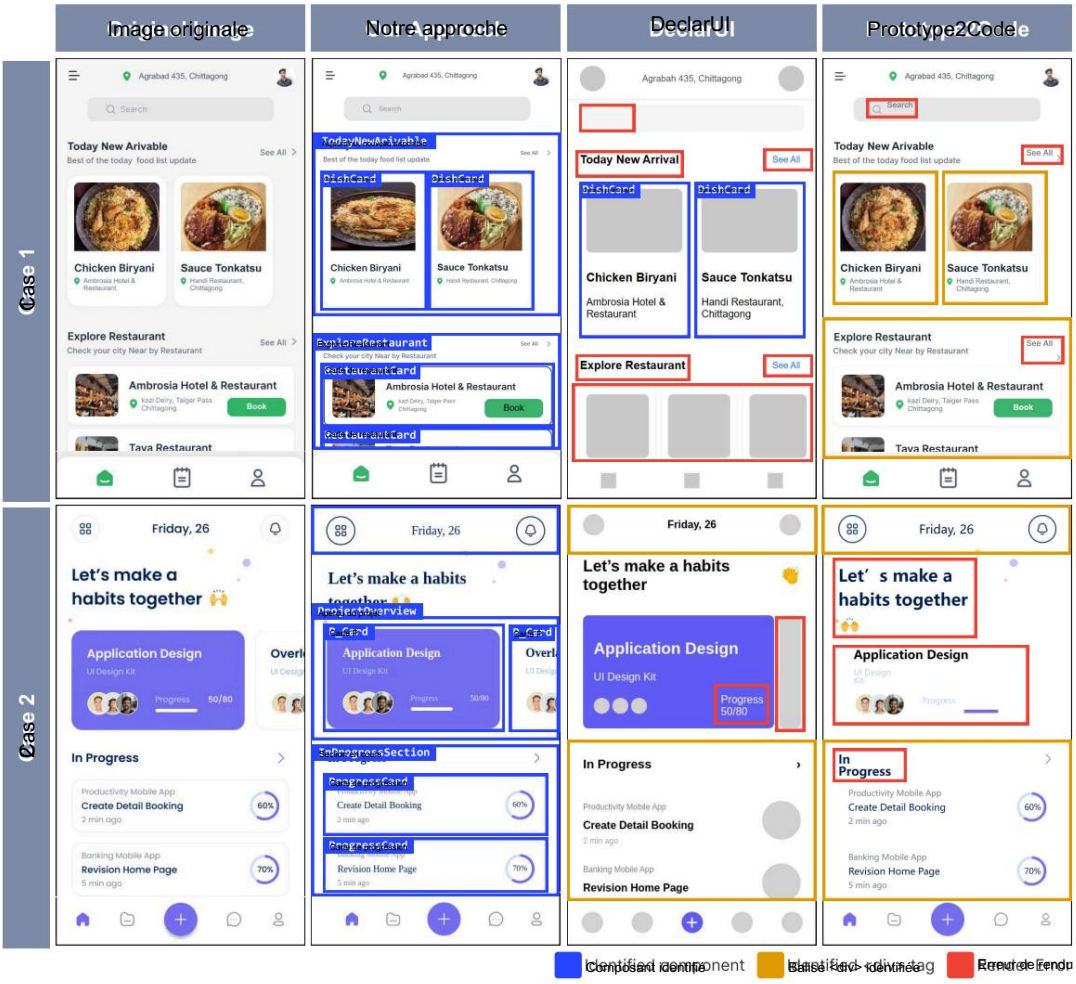


Figure 5 : Exemples illustrant l'efficacité de DesignCoder en matière de rendu et de structure du code, comparée à deux solutions de référence. méthodes.

Les scores CLIP et SSIM, couramment utilisés pour l'évaluation de la similarité visuelle des interfaces utilisateur , peuvent ne pas détecter les différences subtiles. Pour remédier à cela, nous L'erreur quadratique moyenne (MSE) est utilisée pour comparer les images au niveau du pixel. Cependant, une similarité visuelle parfaite n'indique pas nécessairement une qualité élevée. du code généré. Par conséquent, nous intégrons la similarité de la structure du code. métriques. Néanmoins, les structures de code annotées manuellement peuvent ne pas refléter pleinement les meilleures pratiques dans l'implémentation concrète du code. Par conséquent , la robustesse de cette approche nécessite encore une validation plus poussée. dans les travaux futurs.

La menace pesant sur la validité externe réside dans les variations de performance des modèles linéaires mixtes (MLLM) et dans la taille des ensembles de données. Différents MLLM peuvent produire des résultats différents. Les résultats dépendent de l'invite utilisée, ce qui signifie que nos conclusions statistiques peuvent ne pas refléter pleinement les performances optimales de génération de code. de MLLM. De plus, bien que nous ayons sélectionné deux représentants Nous avons utilisé 300 échantillons provenant de différentes sources de données ; la taille de notre ensemble de données pourrait ne pas suffire. toutes les variantes d'interfaces utilisateur d'applications mobiles. Pour atténuer ce problème, à l'avenir, Les recherches devraient élargir à la fois l'échelle et la diversité des échantillons d'interface utilisateur. afin de valider davantage la généralisabilité de l'approche.

Machine Translated by Google

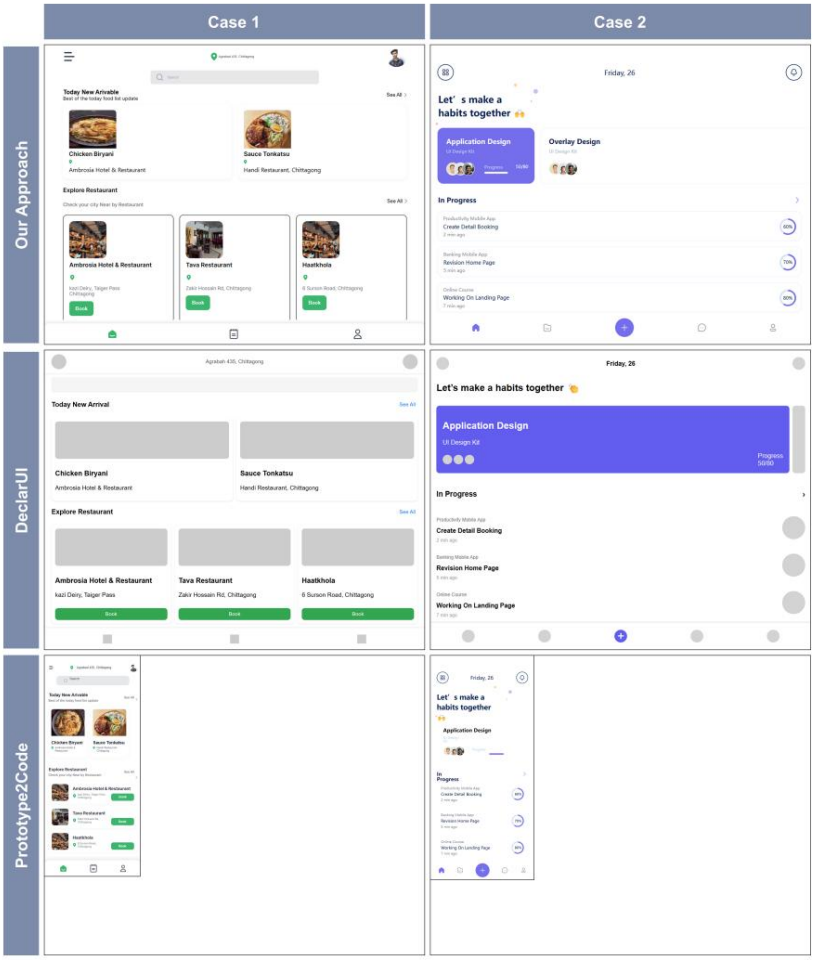


Figure 6 : Exemples illustrant la réactivité de DesignCoder sur différentes tailles d'écran, comparée à deux méthodes de référence.

5 Travaux connexes

5.1 Génération de code à partir d'images Ces dernières années,

de nombreuses études se sont concentrées sur la génération de code d'interface utilisateur à partir d'images. Les approches existantes peuvent être classées en deux grandes catégories : les approches basées sur l'apprentissage profond [6, 7, 9, 24, 33] et les approches basées sur les modèles linéaires mixtes (MLLM) [13, 15, 25, 26, 34]. Pix2Code [6] a été le premier à utiliser des techniques d'apprentissage profond pour générer du code à partir de captures d'écran d'interface graphique. Une autre méthode notable basée sur la vision par ordinateur est Sketch2Code [24], qui convertit des croquis dessinés à la main en HTML. Pour générer des structures d'interface utilisateur complexes et imbriquées, Wu et al. [29] ont introduit le problème d'analyse d'écran, en utilisant Faster R-CNN [23] pour encoder les captures d'écran d'interface utilisateur et en intégrant un modèle LSTM pour générer des graphes hiérarchiques d'interface utilisateur. Cependant, cette approche reste limitée pour la gestion des hiérarchies d'interface utilisateur très complexes avec des structures profondément imbriquées.

5.2 Génération de code à partir de maquettes Dans le

développement front-end industriel, les concepteurs créent des maquettes d'interface utilisateur, tandis que les développeurs front-end les traduisent en code. Cependant , ce processus manuel est souvent long et source d'erreurs. Afin de rationaliser le développement, des techniques de génération de code à partir de maquettes ont émergé, visant à automatiser la génération de code d'interface utilisateur [2, 18, 30].

Comparativement à la génération de code visuelle, les approches intégrant des métadonnées de conception produisent un code plus structuré et maintenable . Cette méthode multimodale soutient mieux le développement front-end industriel en améliorant la cohérence et l'efficacité. Par exemple, Figma2Code [18] utilise une approche d'annotation basée sur les métadonnées pour identifier les composants d'interface graphique (GUI) en vue de la génération de code. Cependant, faute de reconnaissance du type de composants d'interface utilisateur, le code généré ne répond souvent pas aux exigences de développement. Les plateformes d'entreprise telles qu'Imgcook [4] et CodeFun [1] exploitent des modèles de vision pour identifier les composants et les structures de mise en page, puis convertissent les résultats en un langage générique spécifique au domaine. Elles appliquent ensuite diverses règles pour générer du code front-end compatible avec différents frameworks. Bien que ces plateformes soient très performantes pour répondre aux demandes à grande échelle, elles rencontrent des difficultés persistantes telles que la fragmentation des éléments d'interface utilisateur et leur regroupement perceptuel, ce qui nécessite des ajustements manuels supplémentaires du code généré. Prototype2Code [30] améliore les maquettes d'interface utilisateur en intégrant des techniques de lint de conception pour affiner les données d'entrée, permettant ainsi la génération de pages d'interface utilisateur haute fidélité. Cependant, le code généré manque de sémantique des composants, ce qui entraîne une surutilisation des éléments plutôt que des hiérarchies de composa Cette étude porte sur la génération de code structuré et modulaire, ainsi que sur des styles visuels cohérents. Elle intègre la réparation automatisée des programmes.

Machine Translated by Google

En plus des techniques utilisées, notre approche vise à améliorer la facilité d'utilisation du code, sa maintenabilité et son respect des meilleures pratiques de développement front-end.

6 Conclusion

Dans cet article, nous proposons DesignCoder, une nouvelle approche d'autocorrection, sensible à la hiérarchie et guidée par la vision, pour générer du code d'interface utilisateur de haute qualité à partir de maquettes. DesignCoder améliore la compréhension de l'interface utilisateur par les modèles de langage multimodaux (MLLM) grâce à un cadre de raisonnement multimodal basé sur une chaîne de pensée , reconnaissant efficacement les structures imbriquées complexes. Les évaluations empiriques démontrent que DesignCoder surpasse à la fois les méthodes de référence de pointe et les derniers MLLM . Notre approche comble le fossé entre les MLLM et le développement front-end haute fidélité, rendant le code d'interface utilisateur généré par les MLLM utilisable concrètement dans les applications industrielles. Elle accélère les cycles d'itération produit, permettant aux équipes de développement de se concentrer davantage sur les fonctionnalités essentielles et l'innovation produit. Dans nos travaux futurs, nous prévoyons d'intégrer une intention de conception explicite et d'exploiter des bibliothèques de composants pour améliorer encore les comportements dynamiques des composants et la gestion d'état contextuelle.

Références

[1] 2024. Site web Codefun. https://code.fun/. [Consulté le 27 décembre 2024].

[2] 2024. Codia Design to Code. https://codia.ai/design-to-code/. [Consulté le 1er décembre 2024. 27, 2024].

[3] 2024. Figma Software. https://www.figma.com/. [Consulté le 27 décembre 2024].

[4] 2024. Site web Imgcook. https://www.imgcook.com/. [Consulté le 27 décembre 2024].

[5] 2024. Logiciel Sketch. https://www.sketch.com/. [Consulté le 27 décembre 2024].

[6] Tony Beltramelli. 2018. pix2code : Générer du code à partir d'une capture d'écran d'interface utilisateur graphique. Dans les actes du symposium ACM SIGCHI sur l'ingénierie des systèmes informatiques interactifs. 1–6.

[7] Chunyang Chen, Ting Su, Guozhu Meng, Zhenchang Xing et Yang Liu. 2018. De l'image de conception d'interface utilisateur au squelette de l'interface graphique : un traducteur automatique neuronal pour une implémentation d'interface graphique mobile Bootstrap. Dans les actes de la 40e Conférence internationale sur le génie logiciel. 665–676.

[8] Liuqing Chen, Yunnong Chen, Shuhong Xiao, Yaxuan Song, Lingyun Sun, Yankun Zhen, Tingting Zhou et Yanfang Chang. 2024. EGFE : Regroupement de bout en bout d' éléments fragmentés dans les conceptions d'interface utilisateur avec apprentissage multimodal. Dans les actes de la 46e conférence internationale IEEE/ACM sur le génie logiciel. 1–12.

[9] Wen-Yin Chen, Pavol Podstreleny, Wen-Huang Cheng, Yung-Yao Chen et Kai- Lung Hua. 2022. Génération de code à partir d'une interface utilisateur graphique via un modèle encodeur-décodeur basé sur l'attention. Multimedia Systems 28, 1 (2022), 121–130.

[10] Yunnong Chen, Shuhong Xiao, Jiazhi Li, Tingting Zhou, Yanfang Chang, Yankun Zhen, Lingyun Sun et Liuqing Chen. 2024. Regroupement de couches fragmentées dans les conceptions d'interface graphique grâce à l'apprentissage graphique basé sur des informations multimodales. Préimpression arXiv arXiv :2412.05555 (2024).

[11] Yunnong Chen, Yankun Zhen, Chuning Shi, Jiazhi Li, Liuqing Chen, Zejian Li, Lingyun Sun, Tingting Zhou et Yanfang Chang. 2023. Fusion des couches d'interface utilisateur : fusion des couches d'interface utilisateur via l'apprentissage visuel et les limites préalables. Frontières des technologies de l'information et de l'ingénierie électronique 24, 3 (2023), 373-387.

[12] Sidong Feng, Mingyue Yuan, Jieshan Chen, Zhenchang Xing et Chunyang Chen. 2023. Concevoir avec le langage : intention de conception d'interface utilisateur filaire avec de grands modèles de langage génératifs. Préimpression arXiv arXiv :2312.07755 (2023).

[13] Yi Gui, Zhen Li, Yao Wan, Yemin Shi, Hongyu Zhang, Yi Su, Bohua Chen, Dongping Chen, Siyuan Wu, Xing Zhou, et al. 2025. WebCode2M : un ensemble de données du monde réel pour la génération de code à partir de conceptions de pages Web. Dans LA WEBCONFÉRENCE 2025.

[14] Yi Gui, Zhen Li, Yao Wan, Yemin Shi, Hongyu Zhang, Yi Su, Shaoling Dong, Xing Zhou et Wenbin Jiang. 2024. Vision2ui : un ensemble de données du monde réel avec une présentation pour la génération de code à partir de conceptions d'interface utilisateur. Préimpression arXiv arXiv :2404.06369 (2024).

[15] Yi Gui, Yao Wan, Zhen Li, Zhongyi Zhang, Dongping Chen, Hongyu Zhang, Yi Su, Bohua Chen, Xing Zhou, Wenbin Jiang et al. 2025. UICopilot : automatisation de la synthèse de l'interface utilisateur via la génération de code hiérarchique à partir de conceptions de pages Web. Dans LA WEBCONFÉRENCE 2025.

[16] Xue Jiang, Yihong Dong, Yongding Tao, Huanyu Liu, Zhi Jin, Wenpin Jiao et Ge Li. 2024. ROCODE : Intégration du mécanisme de retour en arrière et de l'analyse de programme dans de grands modèles de langage pour la génération de code. Préimpression arXiv arXiv :2411.07112 (2024). Accepté par ICSE 2025.

[17] Alexander Kirillov, Eric Mintun, Nikhila Ravi, Hanzi Mao, Chloe Rolland, Laura Gustafson, Tete Xiao, Spencer Whitehead, Alexander C Berg, Wan-Yen Lo et al. 2023. Segmenter n'importe quoi. Dans les actes de la conférence internationale IEEE/CVF sur la vision par ordinateur. 4015–4026.

[18] Zhu Lin, Feng Yingchaojie, Zhu Hang, Wang Sijia, Zhu Minfeng, Yu Chenhao, Zhang Yuhui, Xu Daxing, Zhao Deming, Feng Yujun et al. 2024. Figma2Code : méthode de génération automatique de code pour les brouillons de conception Figma. Journal de conception assistée par ordinateur et d'infographie (2024).

[19] Haotian Liu, Chunyuan Li, Qingyang Wu et Yong Jae Lee. 2023. Réglage visuel des instructions. Avancées dans les systèmes de traitement de l'information neuronale 36 (2023), 34892–34916.

[20] Shilong Liu, Zhaoyang Zeng, Tianhe Ren, Feng Li, Hao Zhang, Jie Yang, Qing Jiang, Chunyuan Li, Jianwei Yang, Hang Su et al. 2024. Grounding Dino : marier un dinosaure avec une pré-formation à la terre pour la détection d'objets en espace ouvert. Lors de la Conférence européenne sur la vision par ordinateur. Springer, 38-55 ans.

[21] Meta. 2024. React Native. https://reactnative.dev/.

[22] Soumik Mohian et Christoph Csallner. 2020. Doodle2App : Code d'application native par esquisse d'interface utilisateur à main levée. Dans les actes de la 7e conférence internationale IEEE/ACM sur l'ingénierie et les systèmes logiciels mobiles. 81–84.

[23] Shaoqing Ren, Kaiming He, Ross Girshick et Jian Sun. 2016. Faster R-CNN : Vers la détection d'objets en temps réel avec des réseaux de proposition de régions. IEEE transactions on pattern analysis and machine intelligence 39, 6 (2016), 1137–1149.

[24] Alex Robinson. 2019. Sketch2code : Générer un site web à partir d'une maquette papier. arXiv preprint arXiv:1905.13750 (2019).

[25] Chenglei Si, Yanzhe Zhang, Zhengyuan Yang, Ruibo Liu et Diyi Yang. 2024. Design2code : À quel point sommes-nous proches de l'automatisation de l'ingénierie front-end ? arXiv e-prints (2024), arXiv–2403.

[26] Yuxuan Wan, Chaozheng Wang, Yi Dong, Wenxuan Wang, Shuqing Li, Yintong Huo et Michael R Lyu. 2024. Génération automatique du code d'interface utilisateur à partir d'une capture d'écran : une approche basée sur diviser pour régner. Préimpression arXiv arXiv :2406.16386 (2024).

[27] Zhou Wang, Alan C Bovik, Hamid R Sheikh et Eero P Simoncelli. 2004. Évaluation de la qualité d'image : de la visibilité des erreurs à la similarité structurelle. IEEE transactions on image processing 13, 4 (2004), 600–612.

[28] Jason Wu. 2024. UICoder : Optimisation de grands modèles de langage pour la génération de code d'interface utilisateur grâce à un retour d'information automatisé. Dans : Actes de la conférence 2024 de la section nord-américaine de l'Association for Computational Linguistics : Technologies du langage humain (Volume 1 : Articles longs). 7504–7518.

[29] Jason Wu, Xiaoyi Zhang, Jeff Nichols et Jeffrey P Bigham. 2021. Analyse d'écran : vers la rétro-ingénierie des modèles d'interface utilisateur à partir de captures d'écran. Dans : 34e symposium annuel de l'ACM sur les logiciels et technologies d'interface utilisateur. 470–483.

[30] Shuhong Xiao, Yunnong Chen, Jiazhi Li, Liuqing Chen, Lingyun Sun et Tingting Zhou. 2024. Prototype2code : Génération de code front-end de bout en bout à partir de prototypes d'interface utilisateur. Dans : International Design Engineering Technical Conferences and Computers and Information in Engineering Conference, vol. 88353. American Society of Mechanical Engineers, V02BT02A038.

[31] Shuhong Xiao, Yunnong Chen, Yaxuan Song, Liuqing Chen, Lingyun Sun, Yankun Zhen, Yanfang Chang et Tingting Zhou. 2024. Détection de groupe de composants sémantiques de l'interface utilisateur : regroupement d'éléments d'interface utilisateur avec une sémantique similaire dans l'interface utilisateur graphique mobile . Affiche 83 (2024), 102679.

[32] Mulong Xie, Zhenchang Xing, Sidong Feng, Xiwei Xu, Liming Zhu et Chunyang Chen. 2022. Inférence non supervisée, inspirée par la psychologie, de groupes perceptifs de widgets d'interface graphique à partir d'images d'interface graphique. Dans les actes de la 30e conférence et symposium conjoints européens de l'ACM sur le génie logiciel et les fondements du génie logiciel. 332–343.

[33] Yong Xu, Lili Bo, Xiaobing Sun, Bin Li, Jing Jiang et Wei Zhou. 2021. image2emmet : Génération automatique de code à partir d'une image d'interface utilisateur Web. Journal of Software: Evolution and Process 33, 8 (2021), e2369.

[34] Ting Zhou, Yanjie Zhao, Xinyi Hou, Xiaoyu Sun, Kai Chen et Haoyu Wang. 2024. Faire le lien entre la conception et le développement grâce à la génération automatisée de code d'interface utilisateur déclaratif . Prépublication arXiv : arXiv:2409.11667 (2024). Accepté par FSE 2025.